无框架下的 LLMOps 工作流与节点扩展

01. 工作流节点结构设计

在前面的课时中，我们使用 LangGraph 作为工作流底层的框架，并使用提供的 `add_node` 和 `add_edge` 方法来为图结构添加节点和边，如果不使用 LangGraph，作为 AI 应用开发的热门方向，在不使用 LangGraph/Langchain 框架，甚至不使用 Python 的情况下，我们如何设计一套 `工作流` 呢?

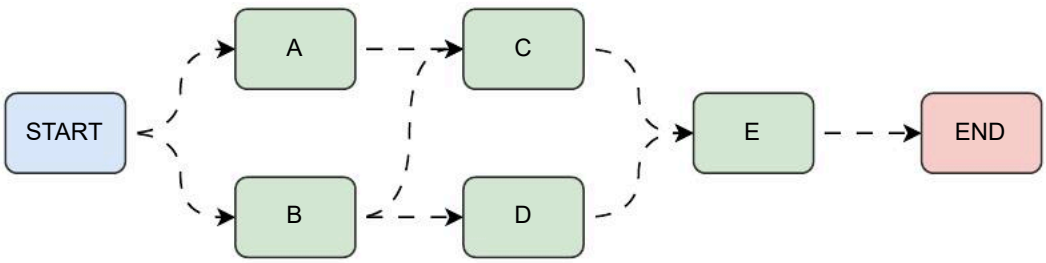
其实 `LangGraph` 中有不少思想值得借鉴，例如 `工作流状态`、`Runnable可运行组件`、`并行节点` 等，所以哪怕在没有 LangGraph 的情况下，我们也可以将节点定义为一个接收 `固定类型参数` 并返回 `固定类型参数` 的函数，节点的 `取数逻辑` 和原先的逻辑保持一致。

工作流除了有节点的具体执行函数之外，还需要定义结构来存储 `节点id与函数的映射关系`、`节点之间的关系图`，节点 id 和函数的映射关系存储相对来说比较简单，使用 `dict(字典)` 类型结构即可，那如果是 `节点之间的关系图` 呢?

```
nodes = {
    "A": function_A,
    "B": function_B,
    "C": function_C,
    "D": function_D,
    "E": function_E,
}
```

其实也可以考虑使用字典，要从字典中顺序取出节点 id，通过递归循环就可以轻松实现，例如有下面一个工作流:

带有并行节点的工作流Demo



可以使用字典来存储该工作流节点的关系，键为 `连线起点节点id`，值为 `连线终点id/ids`，示例如下:

```
edges = {
    "START": ["A", "B"],
    "A": ["C"],
    "B": ["C", "D"],
    "C": ["E"],
    "D": ["E"],
    "E": ["END"],
    "END": [],
}
```

这样在执行工作流时就可以循环遍历这个节点，并记录当前已执行完的节点，并使用递归循环遍历，直到整个工作流执行结束，并且这个思路可以应用在任意的编程语言中，并不限制在 Python 中，JavaScript、golang、Java 等都支持，伪代码如下：

```
# 1.定义节点函数映射
nodes = {
    "A": function_A,
    "B": function_B,
    "C": function_C,
    "D": function_D,
    "E": function_E,
}

# 2.定义节点关系映射
edges = {
    "START": ["A", "B"],
    "A": ["C"],
    "B": ["C", "D"],
    "C": ["E"],
    "D": ["E"],
    "E": ["END"],
    "END": [],
}

# 3.定义递归执行节点函数
def run(activate_nodes: list[UUID]):
    # 4.如果当前激活节点列表长度为1，则取出节点的下一个关联节点
    if len(current_nodes) == 1:
        # 5.取出当前需要执行的节点与函数列表
        running_nodes = nodes.get(current_nodes[0], [])
        running_functions = [nodes.get(running_node) for running_node in
running_nodes]
        # 6.循环执行or并行执行节点函数，并行考虑使用ThreadPoolExecutor来实现
        for running_function in running_functions:
            running_function(状态)
        # 7.执行完成后，递归调用后续节点
        run(running_nodes)
    elif len(current_nodes) > 1:
        # 8.激活节点列表长度大于1，代表存在并行节点，需要循环遍历每个节点并递归调用
        for current_node in current_nodes:
            run([current_node])
    else:
        # 9.如果激活节点长度为空，则代表执行到END节点，结束程序
        return
```

02. 条件分支类节点的扩展思路

在 `FunctionCallAgent` 和 `ReACTAgent` 中，我们使用条件边判断 LLM 节点生成内容为 `AI消息` 还是 `工具调用消息`，从而执行不同的节点，在工作流中也可以通过扩展节点的类型来实现类似的 `条件判断节点`。

条件节点的逻辑很简单，一个 `条件判断节点` 可以拥有多个条件，每个条件可以连接到不同的 `节点` 上，从而执行分支线路，所以在节点中需要存储 `条件信息`、`条件对应的节点信息`，并且识别到节点类型为 `条件节点` 时，为该节点添加一个 `should_continue` 条件路由，将其跳转到不同的节点。

所以该节点设计需要存储的数据+映射的函数如下：

1. `conditions`：条件列表信息，值为字典，字典内存储 `条件类型(if/elif/else)`、`条件信息(包含/等于/大于/小于等)`、`条件id(用来区分出不同的条件)`；
2. `condition_node_map`：条件节点映射，类型为一个字典，键为 `条件id`，值为 `节点id`，代表某个条件连接到某个节点。
3. `condition_function()`：封装一个函数，接收条件、条件映射，并返回各个节点的id，用作条件边。

这样在 `Workflow` 下的 `_build_workflow()` 方法内部，检测到 `node_type` 为 `IF` 类型的时候，不执行普通的 `add_edge()` 函数，而是执行 LangGraph 的 `add_conditional_edges()`，并传递 `条件节点` + `condition_funciton`，这样即可实现在 LLMOps 中扩展条件节点（取数的逻辑不发生变化）。

伪代码：

```
# _build_workflow()
# 1.循环遍历所有节点
for node in nodes:
    # 2.判断节点类型是否为条件节点，完成条件边添加
    if node.node_type == NodeType.IF:
        graph.add_node(node_flag, NodeClasses[NodeType.IF](node_data=node))
        graph.add_conditional_edges(node_flag, condition_function)

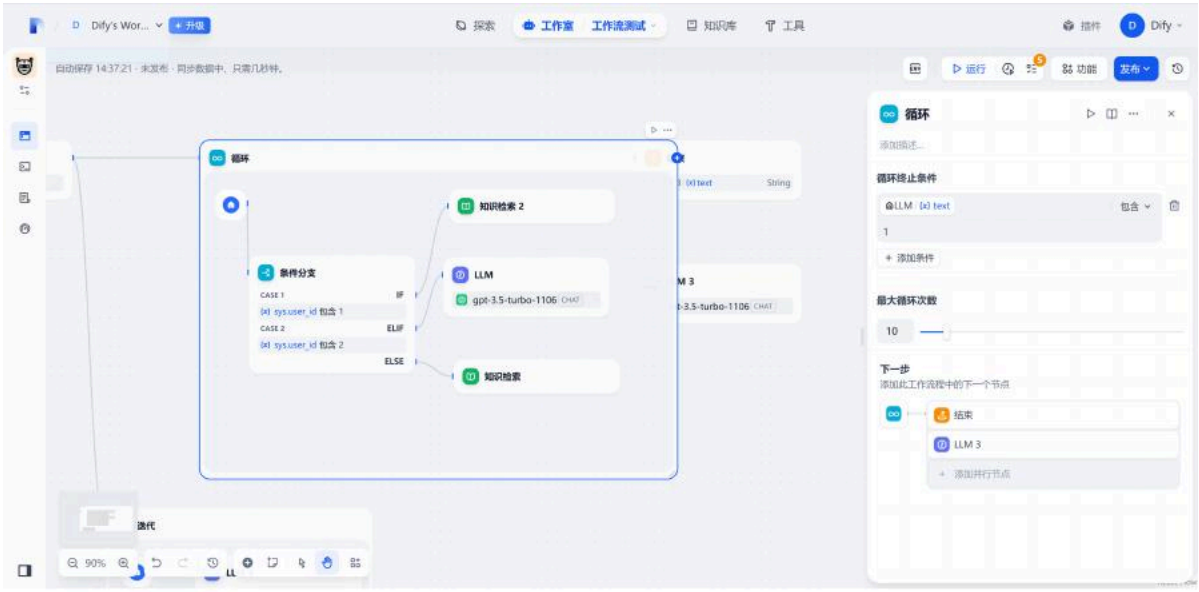
# 3.循环遍历边
for edge in edges:
    # 4.判断边的起点类型是否为IF，如果是则跳过，不添加普通边，在上面的逻辑中已添加条件边
    if edge.source_type == NodeType.IF:
        continue
```

03. 迭代类节点的扩展思路

在 Dify、Coze 中还有一个 `迭代` 类的节点，该节点会接收 `数组类型` 的数据，接收的数据变量为列表，并且该类型节点的输出数据也是列表类型，在 LLMOps 项目中想要扩展该节点，操作会比 `条件节点` 麻烦一些。

首先，在 LLMOps 项目的工作流中，目前设计的 `节点数据` 类型中并不支持 `列表`，所以如果需要新增迭代节点，首先需要扩充 LLMOps 工作流的节点数据类型，新增list类型的数据，

其次，`迭代节点` 会存储一些额外信息，例如 `迭代终止条件`、`最大迭代次数`，并且 `迭代节点` 本身也是一个**包含限定节点的工作流**，如下为 Dify 的 `迭代节点`：



之所以加了一个词 `限定节点`，这是因为在这个 `内嵌工作流` 内部不支持添加 `结束`、`内嵌迭代` 这几个特殊节点，所以设计该节点的思路就是对标 `workflow`+`循环判断`，完成该节点的扩展思路如下：

1. 扩充工作流接收与生成的数据类型，添加 `列表型` 数据；
2. 创建 `迭代节点`，对标 `workflow`，并存储 `nodes` 和 `edges`，这两个数据和 `workflow` 保持一致，并且特殊的节点没法添加（迭代节点、结束节点），在 `迭代节点` 内部构建一个子工作流，同时校验 `nodes` 和 `edges` 的数据；
3. 在节点执行操作时，外层通过执行循环，并且循环次数不超过 `最大循环次数`，同时内部在每一次执行 `自工作流` 结束后，均判断下是否符合 `循环终止条件`，如果符合则跳出循环，并更新状态；
4. 其他操作，例如**添加节点、添加边和其他节点的逻辑保持一致**，没任何差异；

核心伪代码逻辑如下：

```
class IterationNode(BaseNode):
    node_data: IterationNodeData

    def invoke(self, state: xxx) -> xxx:
        # 1.调用函数生成子工作流（逻辑和workflow保持一致）
        workflow = self._build_workflow()

        # 2.执行循环
        result = {}
        for i in range(0, self.node_data.max_iteration_count):
            # 3.调用工作流并传递状态数据
            result_state = workflow.invoke(state)

            # 4.提取结果状态并判断对应条件是否符合
            if self.node_data.condition xxx result_state["output"]:
                # 5.符合条件提取迭代节点输出数据并跳出循环
                result = result_state["xxx"]
                break

            # 6.哪怕不符合条件也提取出数据，这样超过最大迭代次数后，仍然有一个结果
            result = result_state["xxx"]

        return result

    def _build_workflow() -> Runnable:
```

```
    """构建子工作流节点，逻辑和外层workflow保持一致"""
    ...
```